SOFTWARE REQUIREMENT SPECIFICATION (SRS)

For

GEARSHARE – EQUIPMENT SHARING PLATFORM

Prepared by

Nazmul Haque Fahad
ID: 223071019
Department of CSE and CSIT
Shanto-Mariam University of Creative Technology

Submitted to

Shad Al Kaiser
Lecturer
Department of CSE and CSIT
Shanto-Mariam University of Creative Technology

April 4, 2026

Contents

1 Preface **3**

2 Introduction **3**

3 Glossary **3**

4 User Requirements Definition **4**

5 System Architecture **5**

6 User Requirements Specification **6**

7 System Model **7**

8 Data Flow Diagram **11**

9 System Evolution **11**

10 Appendices **12**

11 Index **12**

1 Preface

This document presents the Software Requirement Specification (SRS) for the GearShare Equipment Sharing Platform. The purpose of this document is to clearly define system functionalities, constraints, and performance requirements. It ensures that developers, stakeholders, and users have a common understanding of the system. GearShare is designed to solve the problem of expensive equipment ownership and underutilized resources. Many users need equipment such as cameras, tools, or electronics for short-term use. At the same time, owners often keep such equipment unused. This system connects owners and renters through a secure platform with identity verification, payment systems, and real-time communication. The application ensures trust, safety, and efficiency in the sharing economy.

2 Introduction

A Software Requirement Specification (SRS) provides a complete and structured description of the system to be developed, including its functional and non-functional requirements, constraints, assumptions, and system behavior. It acts as a formal agreement between stakeholders and developers, ensuring that all parties clearly understand the system's objectives, scope, and expected performance.
The SRS also serves as a foundation for system design, implementation, testing, and future maintenance.
The GearShare system is a mobile-based platform built using Flutter and integrated with Supabase backend services. It is designed to support a secure and scalable environment where users can rent and share equipment efficiently. The system leverages modern technologies such as cloud-based databases, real-time synchronization, and secure authentication to ensure reliability, performance, and data integrity. The application enables users to register and authenticate their identity, create and manage equipment listings, search for nearby available items using location-based services, and make bookings based on availability. It also integrates secure payment gateways to handle financial transactions safely.
Additionally, the system provides a real-time chat feature that allows direct communication between equipment owners and renters, improving trust and coordination.
Furthermore, the system includes an administrative module responsible for monitoring user activities, managing listings, handling disputes, and ensuring platform security. The application maintains confidentiality, integrity, and availability of user data through secure backend services and encryption mechanisms. The SRS document includes detailed descriptions of system requirements, overall architecture, data flow, user interactions, and external interfaces. It also outlines system constraints, assumptions, and future enhancements such as AI-based recommendations, smart pricing, and advanced analytics. This document serves as a comprehensive reference throughout the software development lifecycle, ensuring consistency, quality, and successful system delivery.

3 Glossary

Term	**Definition**
Supabase	Backend-as-a-Service providing authentication and database.
Flutter	Framework used for building cross-platform mobile apps.
Equipment	Items available for rent such as cameras or tools.
Owner	User who lists equipment.
Renter	User who books equipment.
Booking	Process of reserving equipment.
Payment Gateway	System for secure financial transactions.
Verification	Identity validation using phone/NID.

4 User Requirements Definition

Table 2: Definition of User Requirements	
Requirement Type	**Definitions**
Functional Requirement	1. Users can register and login securely. 2. Owners can upload equipment details with images. 3. Users can search equipment by category and location. 4. System supports booking and availability checking. 5. System provides secure online payments. 6. Users can communicate via chat. 7. Admin can manage users, listings, and disputes. 8. System provides discount options based on rental duration (e.g., weekly or monthly discounts). 9. Users must complete NID verification for identity validation and trust.
Non-Functional Requirement	10. System must ensure data security and privacy. 11. Response time should be under 3 seconds. 12. Must run on Android and iOS. 13. UI must be intuitive and user-friendly. 14. System must support real-time updates.

5 System Architecture

The GearShare system follows a client-server architecture consisting of:

- **Frontend (Flutter):** Handles user interface and interaction.
- **Backend (Supabase):** Manages authentication, database, and storage.
- **Payment Gateway:** Processes secure transactions.

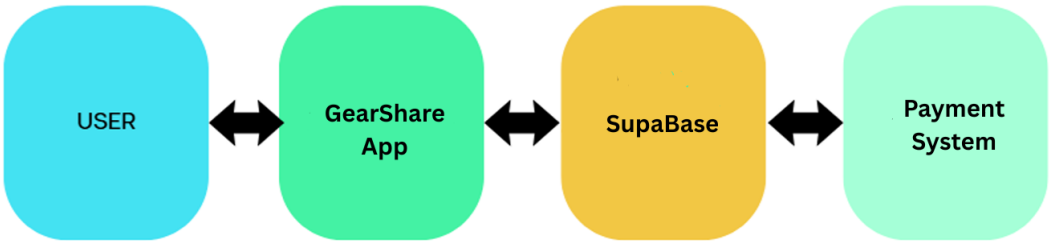

Figure 1: System Architecture of GearShare Platform

Data flows from the user interface to backend services and returns processed results. The architecture ensures scalability, security, and performance.

6 User Requirements Specification

Table 3: Specification of User Requirements	
Requirement Type	**Specifications**
Functional Requirement	1.1 User registration with authentication. 1.2 Equipment listing with images and price. 1.3 Search functionality with filters. 1.4 Booking system with time selection. 1.5 Secure payment integration. 1.6 Chat between renter and owner. 1.7 Rating and review system. 1.8 Admin dashboard for management.
Non-Functional Requirement	2.1 Secure database storage. 2.2 Fast system response. 2.3 Cross-platform compatibility. 2.4 High usability and accessibility. 2.5 Data consistency and integrity.

7 System Model

A. Use Cases

Use cases define the actions or event steps showing the interactions between actors and the system to achieve a goal. The GearShare system use cases are listed in Table 7.

Table 4: List of Use Cases	
Use Case ID	**Use Case Title**
UC1	User Registration
UC2	User Login
UC3	Add Equipment
UC4	Search Equipment
UC5	Book Equipment
UC6	Payment Processing
UC7	NID Verification
UC8	Admin Management

UC1: User Registration

Actors: Owner, Renter, Admin
Preconditions: App installed, internet connected, user does not have an account
Main Success Scenario:

1. Actor opens the app and clicks "Create Account".
2. System displays registration form (Name, Email, Phone, Password).
3. Actor fills in required information.
4. System validates input fields.
5. Actor submits form.
6. System checks if account already exists.
7. System sends verification code.
8. Actor enters code.
9. System creates account in Supabase.

Postcondition: Account successfully created.
Alternative Course:

- Duplicate account → system shows error.
- Verification fails → retry option.

UC2: User Login

Actors: Owner, Renter, Admin
Preconditions: Registered account
Main Success Scenario:

1. Actor opens app and clicks "Login".
2. Enters email/phone and password.
3. System validates credentials.
4. If correct → login successful.

Postcondition: User enters dashboard.
Alternative Course:

- Wrong credentials → error message.

UC3: Add Equipment

Actors: Owner
Preconditions: User logged in
Main Success Scenario:

1. Actor clicks "Add Equipment".
2. System shows form (Name, Category, Price, Image, Location).
3. Actor fills data.
4. System validates input.
5. Actor submits listing.
6. System stores data in Supabase.

Postcondition: Equipment listed.
Alternative Course: Missing data → prompt to complete.

UC4: Search Equipment

Actors: Renter
Preconditions: User logged in
Main Success Scenario:

1. Actor opens search.
2. Enters keyword or selects category.
3. System fetches data from database.
4. Displays nearby available equipment.

Postcondition: Equipment displayed.
Alternative Course: No result → show "No equipment found".

UC5: Book Equipment

Actors: Renter
Preconditions: Equipment available
Main Success Scenario:

1. Actor selects equipment.
2. Chooses rental duration.
3. System calculates price and discount.
4. Actor confirms booking.
5. System updates booking status.

Postcondition: Booking created.
Alternative Course: Already booked → show error.

UC6: Payment Processing

Actors: Renter
Preconditions: Booking confirmed
Main Success Scenario:

1. Actor selects payment method.
2. System connects to payment gateway.
3. Actor completes payment.
4. System confirms transaction.

Postcondition: Payment successful.
Alternative Course: Payment failed → retry.

UC7: NID Verification

Actors: User
Preconditions: Account created
Main Success Scenario:

1. User uploads NID information.
2. System verifies identity.
3. If valid → account marked verified.

Postcondition: Verified user status.
Alternative Course: Invalid → reject.

UC8: Admin Management

Actors: Admin
Preconditions: Admin logged in
Main Success Scenario:

1. Admin views users and listings.
2. Reviews reports.

3. Approves or removes listings.

4. Manages disputes.

Postcondition: System maintained.
Alternative Course: None

B. Use Cases

The use case diagram illustrates the interactions between different users and the GearShare system in a graphical manner.

It demonstrates how key actors including owners, renters, and administrators interact with the system to perform essential operations such as equipment listing, searching based on location and category, booking and payment processing, user verification through NID, and overall system management.

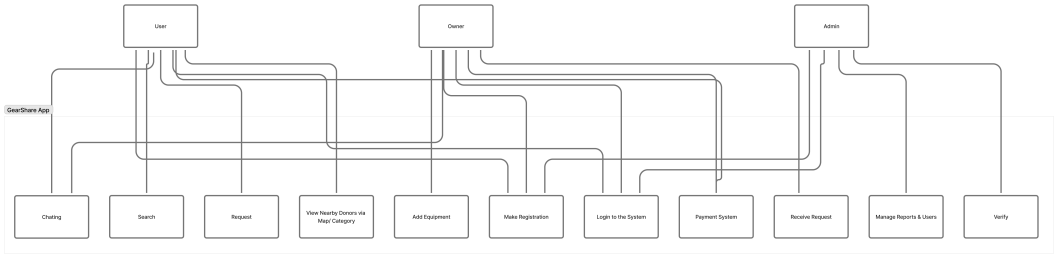

Figure 2: Use Case Diagram of GearShare Platform

8 Data Flow Diagram

The Data Flow Diagram (DFD) represents how data moves within the GearShare system. It illustrates the interaction between users (Owner, Renter, Admin) and the system processes such as registration, equipment listing, booking, payment, and communication.
The diagram shows how data is input, processed, stored, and output across different modules of the system.

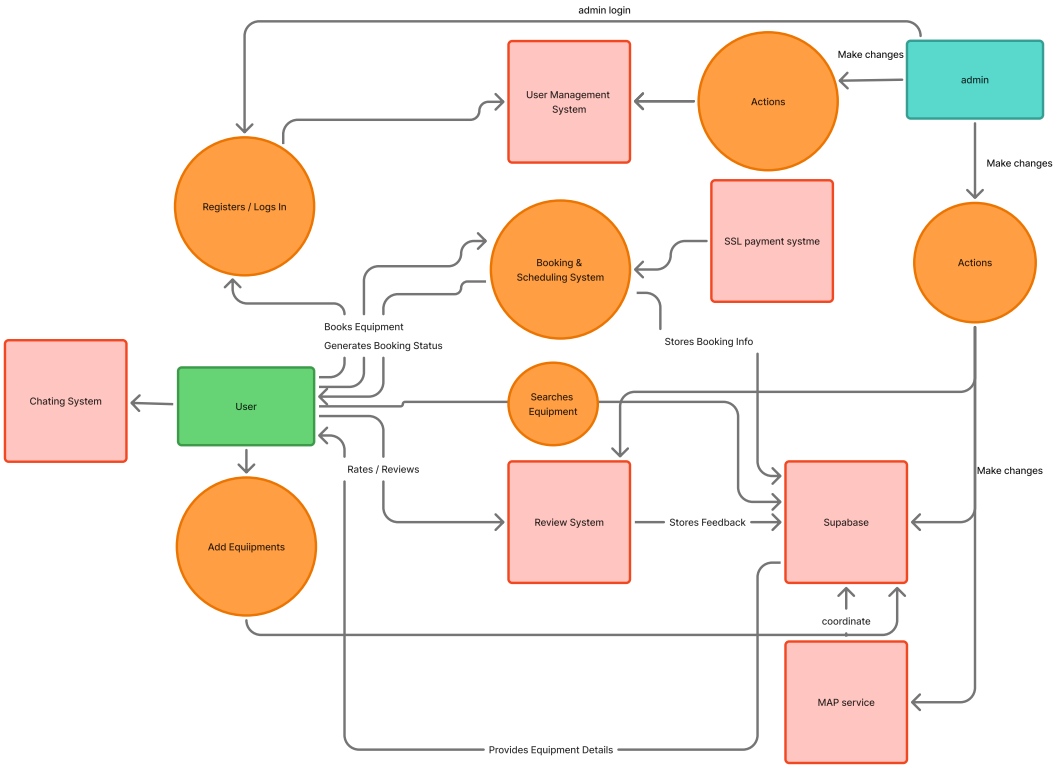

Figure 3: Data Flow Diagram of GearShare Platform

9 System Evolution

Future improvements include:

- AI-based recommendation system
- Smart pricing system
- GPS tracking for equipment
- Insurance integration

10 Appendices

Appendix A : Hardware Specification

Processor	Intel i5 or higher
RAM	8GB minimum
Storage	512GB SSD
OS	Windows/Linux/macOS

Appendix B : Database Specification

Database	Supabase
Security	Authentication + Encryption
Realtime	Supported
Scalability	High

11 Index

A. List of Figures

1	System Architecture	5
2	Use Case Diagram	10
3	Data flow Diagram	11

B. Alphabetic Index

GearShare	All sections
User	Multiple sections
Booking	Section 6
Payment	Section 6

Development Timeline (10 Weeks)

Week	Module/Phase	Tasks / Features
Week 1	Landing & Authentication	Landing page design, Signup page, Login page, Form validation, Supabase authentication
Week 2	User Dashboard & Equipment Upload	User dashboard, Add equipment (name, category, price, description), Image upload, Map integration, Store data in database
Week 3	Search & Booking System	Search by keyword and category, Location-based filtering, Equipment listing UI, Booking system, Owner booking tracking
Week 4	Chat System	Real-time chat, Chat UI, Store messages, New message notification
Week 5	Dummy Payment System	Simulated payment page, Payment options UI (Bkash/Card), Pay Now (always success), Update booking status, Receipt screen
Week 6	Dummy NID Verification	Upload NID image, Input NID details, Simulated verification, Verified badge/status
Week 7	Admin Panel (Core)	Admin dashboard, View users, View equipment, Approve/remove listings, View bookings
Week 8	Admin Advanced Features	Report system, Handle reports, Basic dispute management.
Week 9	Rating & Optimization	Rating system, Review/comments, UI/UX improvement, Bug fixing, Performance optimization
Week 10	Testing & Deployment	Full system testing, Bug fixing, UI polish, Final optimization, Demo and presentation